SKKU 2024 Challenge - Quantum State Tomography

Donghun Jung

November 7, 2024

1 Convention

Problem 1: Quantum States with Few Qubits

The first class of states that we will consider are generic pure quantum states (i.e. there is no restriction on the quantum circuit which created the state), on a small number of qubits. In general, a generic quantum state will require an exponential number of measurements to reconstruct. However, when the number of qubits is small, we can simply pay this exponential cost.

Part I

In this first problem, we have a single qubit quantum state

$$|\psi\rangle = a\,|0\rangle + be^{i\theta}\,|1\rangle\,, \tag{1}$$

where we can treat the unknown parameters a, b and θ as real positive numbers with $|a|^2 + |b|^2 = 1$ and $\theta \in [0, 2\pi)$.

a) Explain how one can determine the values of a, b and θ by measuring the quantum state in different bases.

Start with the given single qubit pure state:

$$|\psi\rangle = a\,|0\rangle + be^{i\theta}\,|1\rangle \tag{2}$$

where a, b, θ are real parameter subject to $a^2 + b^2 = 1$ and $0 \leq \theta < 2\pi$. In order to determine these three parameter, it requires at least three measurement. Our measurement is performed via POVM. There are detectors whose number is equal to the number of qubits. In this scenerio, we perform z-basis projection measurement. Therefore, the POVM becomes $\{|0\rangle\langle 0|\,, |1\rangle\langle 1|\}$, each corresponds to the presense and absence of the signal. Let $\hat{m}_0, \hat{m}_1$ denote the estimated probability of occuring signals. We can

add additional operation U before the measurement and after the computational stage(here, state preparation), we can perform measurement another POVM basis, $\left\{U^\dagger |0\rangle \langle 0| U, U^\dagger |1\rangle \langle 1| U\right\}$. As we should run several circuits, ler U_i denote the additional circuit to i-th circuit.

In first measurement, to determine a, b, we can directly perform measurement without any unitary operation. That is $U_1 = I$, here. Then, we see:

$$\hat{m}_0 \simeq |\langle\psi|0\rangle|^2 \qquad = a^2 \tag{3}$$

$$\hat{m}_1 \simeq |\langle\psi|1\rangle|^2 \qquad = b^2 \tag{4}$$

Therefore, we can estimate,

$$a = \sqrt{\hat{m}_0} \tag{5}$$

$$b = \sqrt{\hat{m}_1} \tag{6}$$

On the other hand, we need two additional measurement. Here let $U_2 = H$, where H is a hadamard gate.

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}. \tag{7}$$

Then, our POVM becomes $\{|+\rangle \langle +|, |-\rangle \langle -|\}$ where $|+\rangle, |-\rangle$ are the eigenstate of Pauli-X matrix whose eigenvalue is $1, -1$ for each. Here, let $\hat{m}_+, \hat{m}_-$ be the estimated probability of occuring signals.

$$\hat{m}_+ \simeq |\langle\psi|+\rangle|^2 \qquad = \left|\frac{a + be^{i\theta}}{2}\right|^2 = \frac{1 + 2ab\cos\theta}{2} \tag{8}$$

$$\hat{m}_- \simeq |\langle\psi|-\rangle|^2 \qquad = \left|\frac{a - be^{i\theta}}{2}\right|^2 = \frac{1 - 2ab\cos\theta}{2} \tag{9}$$

Or,

$$\cos\theta = \frac{\hat{m}_+ - \hat{m}_-}{2ab} \tag{10}$$

Here, it is not enough yet. As there is a degree of freedom choosing a sign of $\sin\theta$. Finally, let $U_3 = SH$ where S is S gate.

$$S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}. \tag{11}$$

Then our POVM becomes $\{|i\rangle \langle i|, |-i\rangle \langle -i|\}$ where $|i\rangle, |-i\rangle$ are the eigenstate of Pauli-Y matrix whose eigenvalue is $1, -1$ for each. Here, let $\hat{m}_i, \hat{m}_{-i}$ be the estimated probability of occuring signals.

$$\hat{m}_i \simeq |\langle\psi|i\rangle|^2 \qquad = \left|\frac{a + be^{i(\theta+\frac{\pi}{2})}}{2}\right|^2 = \frac{1 + 2ab\sin\theta}{2} \tag{12}$$

$$\hat{m}_{-i} \simeq |\langle\psi|-i\rangle|^2 \qquad = \left|\frac{a - be^{i\theta+\frac{\pi}{2}}}{2}\right|^2 = \frac{1 - 2ab\sin\theta}{2} \tag{13}$$

Or,

$$\sin\theta = \frac{\hat{m}_i - \hat{m}_{-i}}{2ab}. \tag{14}$$

Therefore, we can determine a, b and θ.

$$a = \sqrt{\hat{m}_0} \tag{15}$$

$$b = \sqrt{\hat{m}_1} \tag{16}$$

$$\theta = \arcsin\frac{\hat{m}_i - \hat{m}_{-i}}{2ab} = \arccos\frac{\hat{m}_+ - \hat{m}_-}{2ab} \tag{17}$$

b) Write a program in Qiskit which generates this unknown quantum state.

To generate a random single qubit quantum state we use a circuit U_{hidden}, so that $|\psi\rangle = U_{\text{hidden}}|0\rangle$. I used the following provided code, to generate random state.

```
def create_1q_target_circuit(qc_data):
    qc = qiskit.QuantumCircuit(1)
    qc.ry(qc_data[0],0)
    qc.rz(qc_data[1],0)
    return qc
```

As discussed in a), we can determine the state in Eq. 2 via three measurements. To reconstruct the statevector, I implemented the following code.

```
def SingleQubitQST(U_hidden, num_shots:int=100):
  hidden_circuit = U_hidden.copy()

  qc1 = qiskit.QuantumCircuit(1)
  circ1 = hidden_circuit.compose(qc1)
  circ1.measure_all()
  #print(circ1.draw())

  qc2 = qiskit.QuantumCircuit(1)
  qc2.h(0)
  circ2 = hidden_circuit.compose(qc2)
  circ2.measure_all()
  #print(circ2.draw())

  qc3 = qiskit.QuantumCircuit(1)
  qc3.s(0)
```

```
qc3.h(0)
circ3 = hidden_circuit.compose(qc3)
circ3.measure_all()
#print(circ3.draw())

result1 = MeasurementResultHandler(backend
        .run(circ1, shots=num_shots)
        .result().data()["counts"], 1, num_shots)
result2 = MeasurementResultHandler(backend
        .run(circ2, shots=num_shots)
        .result().data()["counts"], 1, num_shots)
result3 = MeasurementResultHandler(backend
        .run(circ3, shots=num_shots)
        .result().data()["counts"], 1, num_shots)

m0 = result1.get("0x0"); m1 = result1.get("0x1")
phi = np.arctan2(np.sqrt(m1), np.sqrt(m0))
phi = phi if phi > 0 else phi + 2*np.pi
statevector = np.array([np.cos(phi),
                        np.sin(phi)],
                        dtype=np.complex64)

m_p = result2.get("0x0"); m_m = result2.get("0x1")
cos = (m_p - m_m)/(2*np.cos(phi)*np.sin(phi))

m_pi = result3.get("0x0"); m_mi = result3.get("0x1")
sin = (-m_pi + m_mi)/(2*np.cos(phi)*np.sin(phi))

temp = np.array([cos, sin])

cos = cos / np.linalg.norm(temp)
sin = sin / np.linalg.norm(temp)
print(circ1.draw())
theta = np.arctan2(sin, cos)
theta = theta if theta > 0 else theta + 2*np.pi

statevector[1] *= np.exp(1j*theta)
circuit_parameter = np.array([2*phi, theta])

return statevector, circuit_parameter
```

Then, we determine the circuit parameters from the reconstructed state vector. Using ZYZ decomposition, we can represent all the unitary operation

on a single qubit. Here, for state preparation, we only need RY and RZ gate[]. The state would be

$$|\psi\rangle = R_z(\beta)R_y(\alpha) = \cos\left(\frac{\alpha}{2}\right)|0\rangle + e^{i\beta}\sin\left(\frac{\alpha}{2}\right)|1\rangle\,. \tag{18}$$

without global phase. Comparing this with the Eq. 2, we can determine circuit parameter α, β.

$$\alpha = 2\arccos a = 2\arcsin b \tag{19}$$

$$\beta = \theta \tag{20}$$

The follwing function performs determining circuit parameters and generating U_{gen}.

```
def SingleQubitCircuit(statevector):
  cos = np.linalg.norm(statevector[0])
  sin = np.linalg.norm(statevector[1])
  phi = np.arctan2(sin,cos)
  theta = (np.log(statevector[1]/
          np.linalg.norm(statevector[1]))*-1j).real

  U_gen = qiskit.QuantumCircuit(1)
  U_gen.ry(2*phi,0)
  U_gen.rz(theta,0)

  return U_gen
```

To evaluate this reconstruction scheme, the following provided code is used.

```
backend2 = Aer.get_backend("statevector_simulator")
def give_score(U_hidden, U_gen):
  qc = U_hidden.compose(U_gen.inverse())
  result = backend2.run(qc).result()
  score = result.get_statevector()[0]
  return np.abs(score)
```

I conducted some simulation with this code.

Part II

We will now increase the complexity by reconstructing a generic two-qubit quantum state.

$$|\psi\rangle = a\,|00\rangle + b\,|01\rangle + c\,|10\rangle + |11\rangle \tag{21}$$

where $|a|^2 + |b|^2 + |c|^2 + |d|^2 = 1$.

a) First, consider the case where all amplitudes are real numbers. Explain how you can measure both the magnitude and sign of the unknown amplitudes, measuring in as few different bases as possible.

b) Now, explain how you would adapt this measurement protocol if the amplitudes a, b, c, d are complex numbers. Note, that we can only reconstruct the wavefunction up to a global phase, so that we can always consider a to be a positive real number. The two-qubit pure state can be written as:

$$|\psi\rangle = a\,|00\rangle + be^{i\theta_b}\,|01\rangle + ce^{i\theta_c}\,|10\rangle + de^{i\theta_b}\,|11\rangle \tag{22}$$

where a, b, c and d are real posivive numbers $\theta_b, \theta_c, \theta_d$ are real numbers subject to $0 \leq \theta_b, \theta_c, \theta_d < 2\pi$ without loss of generality. If the parameters are subjected to real numbers, it seems sufficient to idenify all the parameters a, b, c and d with running two circuits. First, we run circuit without any additional operation, $U = I$. Then our POVM bases are $\{|00\rangle\langle 00|, |01\rangle\langle 01|, |10\rangle\langle 10|, |11\rangle\langle 11|\}$, which directly reproduce a^2, b^2, c^2 and d^2. That is,

$$\hat{m}_{00} \simeq |\langle\psi|00\rangle|^2 = a^2 \tag{23}$$
$$\hat{m}_{01} \simeq |\langle\psi|01\rangle|^2 = b^2 \tag{24}$$
$$\hat{m}_{10} \simeq |\langle\psi|10\rangle|^2 = c^2 \tag{25}$$
$$\hat{m}_{11} \simeq |\langle\psi|11\rangle|^2 = d^2 \tag{26}$$

Next, using Hadamard gate on first qubit, we have:

$$\hat{m}_{+0} \simeq |\langle\psi|+0\rangle|^2 = \left|\frac{a + ce^{i\theta_c}}{\sqrt{2}}\right|^2 \tag{27}$$
$$\hat{m}_{+1} \simeq |\langle\psi|+1\rangle|^2 = \left|\frac{be^{i\theta_b} + de^{i\theta_d}}{\sqrt{2}}\right|^2 \tag{28}$$
$$\hat{m}_{-0} \simeq |\langle\psi|-0\rangle|^2 = \left|\frac{a - ce^{i\theta_c}}{\sqrt{2}}\right|^2 \tag{29}$$
$$\hat{m}_{-1} \simeq |\langle\psi|-1\rangle|^2 = \left|\frac{be^{i\theta_b} - de^{i\theta_d}}{\sqrt{2}}\right|^2 \tag{30}$$

Then, we see:

$$\cos\theta_c = \frac{\hat{m}_{+0} - \hat{m}_{-0}}{2} \tag{31}$$
$$\cos(\theta_b - \theta_d) = \frac{\hat{m}_{+1} - \hat{m}_{-1}}{2} \tag{32}$$
$$\tag{33}$$

With Hadamard gate and S gate on first qubit, we see:

$$\hat{m}_{i0} \simeq |\langle\psi|i0\rangle|^2 = \left|\frac{a + ce^{i\theta_c + \frac{\pi}{2}}}{\sqrt{2}}\right|^2 \tag{34}$$

$$\hat{m}_{i1} \simeq |\langle\psi|i1\rangle|^2 = \left|\frac{be^{i\theta_b} + de^{i\theta_d + \frac{\pi}{2}}}{\sqrt{2}}\right|^2 \tag{35}$$

$$\hat{m}_{-i0} \simeq |\langle\psi|-i0\rangle|^2 = \left|\frac{a - ce^{i\theta_c - \frac{\pi}{2}}}{\sqrt{2}}\right|^2 \tag{36}$$

$$\hat{m}_{-i1} \simeq |\langle\psi|-i1\rangle|^2 = \left|\frac{be^{i\theta_b} - de^{i\theta_d - \frac{\pi}{2}}}{\sqrt{2}}\right|^2 \tag{37}$$

Then,

$$\sin\theta_c = \frac{\hat{m}_{+0} - \hat{m}_{-0}}{2} \tag{38}$$

$$\sin(\theta_b - \theta_d) = \frac{\hat{m}_{+1} - \hat{m}_{-1}}{2} \tag{39}$$

$$\tag{40}$$

Also, using Hadamard gate on second qubit, we have:

$$\hat{m}_{0+} \simeq |\langle\psi|0+\rangle|^2 = \left|\frac{a + be^{i\theta_b}}{\sqrt{2}}\right|^2 \tag{41}$$

$$\hat{m}_{0-} \simeq |\langle\psi|0-\rangle|^2 = \left|\frac{a - be^{i\theta_b}}{\sqrt{2}}\right|^2 \tag{42}$$

$$\hat{m}_{1+} \simeq |\langle\psi|1+\rangle|^2 = \left|\frac{ce^{i\theta_c} + de^{i\theta_d}}{\sqrt{2}}\right|^2 \tag{43}$$

$$\hat{m}_{1-} \simeq |\langle\psi|1-\rangle|^2 = \left|\frac{ce^{i\theta_c} - de^{i\theta_d}}{\sqrt{2}}\right|^2 \tag{44}$$

Then, we see:

$$\cos\theta_b = \frac{\hat{m}_{0+} - \hat{m}_{0-}}{2} \tag{45}$$

$$\cos(\theta_c - \theta_d) = \frac{\hat{m}_{1+} - \hat{m}_{1-}}{2} \tag{46}$$

$$\tag{47}$$

With Hadamard gate and S gate on second qubit, we see:

$$\hat{m}_{0i} \simeq |\langle\psi|0i\rangle|^2 = \left|\frac{a + be^{i\theta_b + \frac{\pi}{2}}}{\sqrt{2}}\right|^2 \tag{48}$$

$$\hat{m}_{1i} \simeq |\langle\psi|1i\rangle|^2 = \left|\frac{a + be^{i\theta_b + \frac{\pi}{2}}}{\sqrt{2}}\right|^2 \tag{49}$$

$$\hat{m}_{0-i} \simeq |\langle\psi|0-i\rangle|^2 = \left|\frac{ce^{i\theta_c} + de^{i\theta_d - \frac{\pi}{2}}}{\sqrt{2}}\right|^2 \tag{50}$$

$$\hat{m}_{0-i} \simeq |\langle\psi|0-i\rangle|^2 = \left|\frac{ce^{i\theta_c} - de^{i\theta_d - \frac{\pi}{2}}}{\sqrt{2}}\right|^2 \tag{51}$$

Then,

$$\sin\theta_b = \frac{\hat{m}_{+0} - \hat{m}_{-0}}{2} \tag{52}$$

$$\sin(\theta_c - \theta_d) = \frac{\hat{m}_{+1} - \hat{m}_{-1}}{2} \tag{53}$$

$$\tag{54}$$

Now we can determine $\theta_b, \theta_c, \theta_d$.

Error Handling

c) Now write a program in Qiskit which will generate this unknown quantum state. Use the same format as for part I. To generate random state the following provided code is used:

```
def create_2q_target_circuit(qc_data):
    qc = qiskit.QuantumCircuit(2)
    for i in range(2):
        qc.ry(qc_data[2*i+1],i)
        qc.rz(qc_data[2*i+1],i)
    qc.cz(0,1)
    qc.rz(qc_data[4],1)
    qc.cx(0,1)
    for i in range(2):
        qc.ry(qc_data[2*i+5],i)
        qc.rz(qc_data[2*i+5],i)
    return qc
```

As discussed in b), we can reconstruct state vector with 5 measurements. I implemented the following code to reconstruct state vector.

```
def TwoQubitQST(U_hidden, num_shots:int=100):
  hidden_circuit = U_hidden.copy()
```

Now we should determine circuit parameters and ansatz. It is known that one CNOT gate is enough to prepare two qubit state. So I used the following circuit ansatz.

For bipartite pure state $|\psi\rangle$, there exist orthonormal state $|i\rangle$ the state for the first party, $|j\rangle$ the state for the other party such that

$$|\psi\rangle = \sum_i \lambda_i |i_1\rangle \otimes |i_2\rangle \tag{55}$$

where λ_i is non-negative real numbers with $\sum_i \lambda_i^2 = 1$. It is known to schmidt decomposition. By the singular value decomposition, $\lambda = UDV$, where D is a diagonal matrix with non-negative elements, and U, V are unitary matrices. Then,

$$|\psi\rangle = \sum_{i,j,k} U_{ji} D_{ii} V_{ik} |j\rangle |k\rangle . \tag{56}$$

Defining $|i_1\rangle = \sum_j U_{ji} |j\rangle$, $|i_2\rangle = \sum_j V_{ik} |k\rangle$ and $\lambda_i = d_{ii}$, we have the same form of Eq. 55. But here, for convenience in circuit construction, make $\langle 0|i_1\rangle$ and $\langle 0|i_2\rangle$ be real number. It can be done by extracting global phase out, then λ_i is no longer real number. Also, we can extract global phase out, $\lambda_i' = \lambda_i \frac{|\lambda_0|}{\lambda_0}$ so that λ_0' to be real number.

Here is a strategy to make circuit ansatz and determine circuit parameter. From the initial state, $|\psi\rangle = |00\rangle$, by using local unitary gate on the first qubit, we have:

$$|\psi\rangle \to \lambda_0' |00\rangle + \lambda_1' |01\rangle \tag{57}$$

Then, apply CNOT gate.

$$|\psi\rangle \to \lambda_0' |00\rangle + \lambda_1' |11\rangle \tag{58}$$

Then, applying local unitary operation U_0, U_1 on each qubit, such that

$$U_0 |0\rangle \to |0\rangle \qquad U_0 |1\rangle \to |0\rangle \tag{59}$$

$$U_1 |0\rangle \to |0\rangle \qquad U_1 |1\rangle \to |0\rangle \tag{60}$$

Finally, the state becomes our reconstructed state in schmidt decomposition form in Eq. 55.

Now local unitary matters. First, we need to make unitary, $|0_1\rangle \to \lambda_0' |0\rangle + \lambda_1' |1\rangle$. This can be done YZ decomposition, as done in Part I.

$$R_z(\beta_1) R_y(\alpha_1) |0\rangle = \cos\frac{\alpha}{2} |0\rangle + e^{i\beta} \sin\frac{\alpha}{2} \tag{61}$$

Then, we have

$$\alpha_1 = 2\arccos\lambda_0' = 2\arcsin|\lambda_1'| \tag{62}$$

$$\beta_1 = -i\ln\frac{\lambda_1}{|\lambda_1|}. \tag{63}$$

Next, two local unitary U_1, U_2 should be implemented. However, as $|0\rangle$ and $|1\rangle$ orthonormal basis, it can also implemented as if we are doing state preparation, by using Ry, Rz gate. We made $\langle 0|i_1\rangle$ and $\langle 0|i_2\rangle$ be real number, so

$$\alpha_2 = 2\arccos\langle 0|i_1\rangle \tag{64}$$

$$\beta_2 = -i\ln\frac{\langle 1|i_1\rangle}{|\langle 1|i_1\rangle|} \tag{65}$$

$$\alpha_3 = 2\arccos\langle 0|i_2\rangle \tag{66}$$

$$\beta_3 = -i\ln\frac{\langle 1|i_2\rangle}{|\langle 1|i_2\rangle|} \tag{67}$$

Therefore, my algorithm procedure is as following:

Phase 1. Perform Schmidt decomposition on reconstructed vector. Numpy has schmidt decomposition method. The column vector in U becomes the basis $|i_1\rangle$ and row vector in $V^\dagger$ are the basis $|i_2\rangle$.

Phase 2. Extract global phase for all schmidt decomposed basis so that the first element of the basis vector to be real. It can done by dividing global phase to state vector and multiplying it to λ_i. And extract global phase one more time, so that λ_0' be real.

Phase 3. Calculate circuit parameters, by using Eq. 63 and Eq. 67.

Phase 4. Construct circuit.

Exception

Part III

Describe how you could extend the protocol developed in part's I) and II) to a general n qubit quantum state. How many different measurements would you need to apply?

The reconstruction of quantum state constitutes two parts: state reconstruction and circuit construction. An arbitrary n-qubit pure state can be

written as:

$$|\psi\rangle = \sum_{i=0}^{2^n-1} a_i e^{\theta_i} |i\rangle \tag{68}$$

where i denotes bit stirng, a_i are real positive number under the constraint $\sum_i |a_i|^2 = 1$, $0 \leq \theta_i < 2\pi$ and $\theta_0 = 0$ without loss of generality. First, state reconstruction can be done two step, extract amplitudes and measure phase. The amplitudes a_i can be easily extracted by measuring on a basis $\{|0\rangle\langle 0|, |1\rangle\langle 1|, \cdots, |2^n-1\rangle\langle 2^n-1|\}$, that is, without any additional operation before measurement and after state preparation. The phase difference between $|i\rangle$ and $|j\rangle$, where hamming distance between i and j is 1, can be conducted by using additional unitary operation on i-th qubit, $U = H_i$ and $U = S_i H_i$. Then we can determine phase difference $\theta_i - \theta_j$ as done in Part I and Part II. In order to determine whole phase difference, such measurement, with additional unitary operation on i-th qubit, $U = H_i$ and $U = S_i H_i$, should be done for all qubits. Therefore, we need $2n+1$ circuits to perform QST on n-qubit pure state.

Once state vector is reconstructed, we can determine circuit ansatz and parameters. Up to three qubits, there are well known ansatz to perpare arbitrary states. One and two qubit cases are discussed above Part I and Part II. For three qubits, it is well known that three CNOT gate is sufficient for preparation of arbitrary three-qubit pure state. Or this reference can be another example[**?**]. For $n \geq 4$, here are strategy[**?**].

Phase 1 If n is an even number, $n = 2k$, then, by Schmidt decomposition, we have:

$$|\psi\rangle = \sum_{i=0}^{2^k-1} \alpha_i |i_1\rangle |i_2\rangle \tag{69}$$

Phase 2 In a similar way to the Part II, apply local unitary gate to initial state, $ket\psi = ket0^{\otimes n}$, such that

$$ket\psi \to \left(\sum_{i=0}^{2^k-1} \alpha_i |i\rangle \right) \otimes |0\rangle^{\otimes k}. \tag{70}$$

Phase 3 Next, we perform k CNOT gates with qubit i(from $i = 0$ to $i = k-1$) as controlled qubit and qubit $i+k$ as target qubit.

$$|\psi\rangle \to \sum_{i=0}^{2^k-1} \alpha_i |i\rangle |i\rangle \tag{71}$$

Phase 4 Then, apply k-qubit unitary operation U_1, U_2 on each party, respectively such that

$$U_1 |i\rangle = |i\rangle \qquad\qquad U_2 |i\rangle = |i\rangle \tag{72}$$

In the case of $n = 2k - 1$, we divide the system into bipartite state by $k - 1$ qubits and k qubits.

Phase 1 From schmidt decomposition, we have:

$$|\psi\rangle = \sum_{i=0}^{2^{k-1}-1} \alpha_i |i_1\rangle |i_2\rangle . \tag{73}$$

Phase 2 Apply local unitary gate to initial state, $ket\psi = ket0^{\otimes n}$, such that

$$ket\psi \to \left(\sum_{i=0}^{2^{k-1}-1} \alpha_i |i\rangle \right) \otimes |0\rangle^{\otimes k} . \tag{74}$$

Phase 3 Apply k CNOT gates with qubit i(from $i = 0$ to $i = k - 2$) as controlled qubit and qubit $i + k$ as target qubit.

$$|\psi\rangle \to \sum_{i=0}^{2^{k-1}-1} \alpha_i |i\rangle |i\rangle \tag{75}$$

Here the last qubit state remains $|0\rangle$ at this stage, here comes the difference.

Phase 4 Then, apply $k - 1$-qubit, k-qubit unitary operation U_1, U_2 on each party, respectively such that

$$U_1 |i\rangle = |i\rangle \qquad\qquad U_2 |i\rangle = |i\rangle \tag{76}$$

The Phase 1, 2 corresponds to the state preparation. So we can perform state preparation reculsively. The problem is making k-qubit unitary operation. This procedure is well introduced in the follwing reference[**?**]. To breifly explain, there exist a representation for k-qubit unitary operation $U \in \mathbb{C}^{2^k \times 2^k}$

$$U = \begin{pmatrix} A_0 & 0 \\ 0 & A_1 \end{pmatrix} \begin{pmatrix} C & -S \\ S & C \end{pmatrix} \begin{pmatrix} B_0 & 0 \\ 0 & B_1 \end{pmatrix} \tag{77}$$

where $A_0, A_1, B_0, B_1 \in \mathbb{C}^{2^k \times 2^k}$ are unitary matrices and C, S are real diagonal matrics such that $C^2 + S^2 = 1$. This decomposition is known as Cosine-Sine Decomposition(CSD). CSD can be performed in following ways. U is partitioned into four equally sized blocks:

$$U = \begin{pmatrix} U_{00} & U_{01} \\ U_{10} & U_{11} \end{pmatrix} \tag{78}$$

Then there exist unitary matrices A_0, A_1, B_0, B_1 such that

$$U = \begin{pmatrix} A_0 & 0 \\ 0 & A_1 \end{pmatrix} \begin{pmatrix} C & -S \\ S & C \end{pmatrix} \begin{pmatrix} B_0 & 0 \\ 0 & B_1 \end{pmatrix}. \tag{79}$$

And it is actually performing four singular value decomposition.

$$U_{00} = A_0 C B_0^\dagger \tag{80}$$

$$U_{01} = A_0 S B_1^\dagger \tag{81}$$

$$U_{10} = A_1 (-S) B_0^\dagger \tag{82}$$

$$U_{11} = A_1 C B_1^\dagger \tag{83}$$

Then,

$$U = |0\rangle\langle 0| \otimes A_0 + |0\rangle\langle 0| A_1 U = |0\rangle\langle 0| \otimes B_0 + |0\rangle\langle 0| B_1 \tag{84}$$

$I \otimes |0\rangle\langle 0|_i + R_y \otimes |1\rangle\langle 1|$

Problem 2 : Graph States

In some cases, one can reconstruct a quantum state using only a limited number of measurements, regardless of how many qubits are in the quantum system. One of these cases is when the quantum states are generated using only "Clifford gates" (these are essentially quantum circuits that use only H,S, CNOT and CZ gates). In this problem, we will perform quantum state tomography on a subset of Clifford states known as "graph states". A graph state on n qubits is any state which can be generated using a circuit of the form

$$|\psi\rangle = \left(\Pi_{(i,j)\in G} CZ_{i,j}\right) H^{\otimes n} |0\rangle \tag{85}$$

Where CZ_{ij} is a controlled-Z gate applied on qubits i and j, and (i,j) are edges of a hidden graph G. Different graphs, G, create different graph states. Our goal is to determine generate the graph state, $|\psi\rangle$, without knowing the specific graph which was used.

a) Look into the properties of graph states. Graph states are stabilizer states with a specific structure. Describe the key properties of stabilizer states. Using the stabilizer properties of graph states, describe how one can perform tomography on graph states. Graph states are a class of stabilizer states with a defined structure. Stabilizer states for n-qubit systems are uniquely characterized by n stabilizers, each being an N-fold tensor product of Pauli operators (up to a factor of $\{\pm 1, \pm i\}$). A stabilizer is an operator under which the state remains invariant, making the state a simultaneous $+1$ eigenstate of all N stabilizer:

$$S_i |\psi\rangle = |\psi\rangle \tag{86}$$

For graph states, the stabilizer associated with each vertex i has the form:

$$S_i = X_i \Pi_{(i,j)\in G} Z_j \tag{87}$$

where each stabilizer S_i corresponds to a specific qubit i and its neighbors in Graph G.

Here is an important remark. a graph state is a simultaneous $+1$ eigenstate of its stabilizers, meaning it can be represented as a linear combination of the stabilizer eigenbasis. The stabilizer eigenbasis takes the form $|\pm\rangle_i \bigotimes_{j\neq i} |b_j\rangle$ with condition $b_i \bigoplus_{j\in(i,j)} b_j = 0$, imposed by the $+1$ eigenvalue requirement(where $+, -$ correspond to $0, 1$).

To perform measurements in this basis for a given vertex i, apply a Hadamard gate H to qubit i before the measurement. After collecting multiple measurement outcomes, identify the set of j such that $b_i \bigoplus j \in (i,j) \oplus b_j = 0$. By repeating this process for each qubit, we can reconstruct the graph of the n-qubit system by running the circuit n times.

```
def generate_clifford_circ(num_qubits, depth):
    U_hidden = qiskit.QuantumCircuit(num_qubits)
    for i in range(num_qubits):
        U_hidden.h(i)
    count = 0
    s1s2 = {-1,-1}
    while(count < depth):
        site_i = int(np.random.rand()*num_qubits)
        site_j = int(np.random.rand()*num_qubits)
        if(site_i == site_j or {site_i, site_j} == s1s2):
            continue
        print(site_i, site_j, count)
        count += 1
        s1s2 = {site_i, site_j}
        U_hidden.cz(site_i, site_j)
    return U_hidden
```

b) You may use the following code snippet to generate a random graph state

You may set num_qubits ¡ 20. Following the format of Problem 1, measure U_{hidden} in ¡= 20 bases by appendix quantum circuits to U_{hidden} and taking no more than 100 shots per basis. Then create a new circuit U_{gen} such that $U_{\text{gen}}^{\dagger} U_{\text{hidden}} |0\rangle = |0\rangle$. As outlined in part a), I implemented the following algorithm:

Phase 1 Apply a Hadamard gate to the i-th site after generating the graph state and before measurement.

Phase 2 Extract the resulting bit strings from the measurements.

Phase 3 Identify the set of j such that $b_i \bigoplus j \in (i,j) \oplus b_j = 0$. Checking all possible sets of (i,j) is feasible since the total number of combinations to check is $n2^{n-1}$, which amounts to at most $20 \times 2^19 \simeq 10485760$ per graph state.

Here are the implemented code:

```
import itertools

def GraphStateQST(U_hidden, num_qubits, num_shots=100):
  hidden_circuit = U_hidden.copy()

  graph = [] # G: set of (i,j)
  for i in range(num_qubits):
    # Phase 1
    qc = qiskit.QuantumCircuit(num_qubits)
    qc.h(i)
    circ = hidden_circuit.compose(qc)
    circ.measure_all()

    # Phase 2
    # Extract bit string
    result = backend.run(circ, shots=num_shots)
             .result().data()["counts"]
    result = [int(i,16) for i in result.keys()]

    # Phase 3
    # Generating possible set of (i,j)
    qubit_index = [idx for idx in range(num_qubits)]
    qubit_index.pop(i)

    for k in range(1, num_qubits):
      for comb in itertools.combinations(qubit_index, k):
        isConnected = xor_bits_at_positions(result, comb, i)
        if isConnected:
          for edge in comb:
            graph.append({i, edge}) \\
        if {i, edge} not in graph else None
```

```
  return graph
```

Here are the other functions, one to verify $b_i \bigoplus j \in (i,j) \oplus b_j = 0$ and the other for generating graph state from the given graph, represented by U_{gen}.

```
# Phase 3
def xor_bits_at_positions(nums, positions, base_location):
  for num in nums:
    # Start with 0 for XOR accumulation
    result = (num >> base_location) & 1

    # XOR each specified bit position
    for pos in positions:
      # Extract the bit at the current position
    # and XOR it with the result
      result ^= (num >> pos) & 1

    if result == 0:
      continue
    else:
      return False

  return True

# Given graph G genreate graph state.
def GraphState(num_qubits, graph):
  U_gen = qiskit.QuantumCircuit(num_qubits)
  for i in range(num_qubits):
    U_gen.h(i)

  for vertex in graph:
    temp = list(vertex)
    site_i = temp[0]
    site_j = temp[1]
    U_gen.cz(site_i, site_j)

  return U_gen
```

Problem 3 : Low Depth Brickwork Circuits